Guía 3 · Subir la app a Google Apps Script

Tablero de Indicadores · Panadería San Nicolás — publicación como aplicación web para la entrega

Resultado	Un enlace de Google (`https://script.google.com/macros/s/…/exec`) que abre la app: el prototipo v0 el 27 de octubre y el prototipo final el 29, con la misma URL.
Archivos	Carpeta **Backend/** del repositorio: `Code.gs` , `Index.html` y `appsscript.json`
Necesitas	Una cuenta de Google y un computador (el editor de Apps Script no funciona bien en el celular)
Tiempo	15 minutos

Antes de empezar: qué cambia y qué no

El docente rechazó la primera versión porque el código vivía solo en Apps Script, usaba Google Sheets y no generaba el tablero con un archivo nuevo. Esta publicación **es distinta**, y conviene decirlo así en la sustentación:

	Primera versión (rechazada)	Esta publicación
Código fuente	Solo en el editor de Apps Script	En GitHub (`Frontend/`); `Backend/Index.html` se **genera** desde ahí
Datos	Google Sheets	El Excel que carga el usuario, procesado en su navegador
Qué hace Google	Formularios y guardar en hojas	Solo entrega la página (`doGet` en `Code.gs`)
Si Google falla	La app no funciona	La misma app sigue en GitHub Pages y sin internet

La justificación está en `Docs/Decisiones_Tecnicas.md` .

Cuándo se publica

Momento	Historia	Qué se hace
Martes 27 oct.	**HU-06 · prototipo v0**	Primera publicación (pasos 1 a 6). Muestra el OEE del horno con A, R y Q.
Jueves 29 oct.	**HU-16 · prototipo final**	Se reemplaza `Index` y se crea una nueva versión de la misma implementación (sección «Actualizar»). La URL no cambia.

> El `Backend/Index.html` de cada momento ya viene generado en el paquete de la HU-06 y en el de la HU-16. Si se quiere regenerar: `python Backend/construir_index.py` desde la raíz del repositorio.

Opción A · Desde el navegador (recomendada)

1. Crear el proyecto

1. Entrar a **script.google.com** con la cuenta de Google del equipo.
2. Clic en **Nuevo proyecto**.
3. Arriba, donde dice *Proyecto sin título*, escribir **Tablero Pan Trenza**.

2. Pegar Code.gs

1. En el editor ya existe el archivo **Código.gs** (o *Code.gs*). Borrar todo su contenido.
2. Abrir `Backend/Code.gs` del repositorio (con Bloc de notas o Cursor), copiar todo y pegarlo.

```
function doGet() {
  return HtmlService.createHtmlOutputFromFile('Index')
    .setTitle('Tablero Pan Trenza · Panadería San Nicolás')
    .addMetaTag('viewport', 'width=device-width, initial-scale=1')
    .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}
```

3. Crear el archivo Index

1. A la izquierda, junto a **Archivos**, clic en **+ → HTML**.
2. Nombrarlo exactamente **Index** (con I mayúscula y sin «.html»; el editor lo agrega solo).
3. Borrar el contenido que trae.
4. Abrir `Backend/Index.html` del repositorio con Cursor o Bloc de notas, seleccionar todo (`Ctrl`+`A`), copiar (`Ctrl`+`C`) y pegar en el editor.
5. Guardar con `Ctrl`+`S` (ícono de disquete).

> `Index.html` pesa unos 120 KB porque incluye los datos de ejemplo de septiembre. Es normal que el editor tarde unos segundos al pegar.

4. Implementar como aplicación web

1. Arriba a la derecha: **Implementar → Nueva implementación**.
2. En el engranaje de *Seleccionar tipo*, elegir **Aplicación web**.
3. Descripción: `Prototipo v0 · HU-06`
4. **Ejecutar como:** Yo (su correo).
5. **Quién tiene acceso: Cualquier usuario** (así el docente lo abre sin iniciar sesión).
6. Clic en **Implementar**. Si pide autorización: **Autorizar acceso** → elegir la cuenta → si aparece «Google no verificó esta app», clic en **Configuración avanzada → Ir a Tablero Pan Trenza (no seguro)** → **Permitir**. Es su propio script; no pide permisos sobre Drive ni Sheets.
7. Copiar la **URL de la aplicación web** (termina en `/exec`).

5. Probar

1. Abrir la URL en una ventana de incógnito (así se ve como la verá el docente).

2. **Usar datos de ejemplo**: OEE **67,7 %** (v0 y final). En el final además: SLA **92,1 %**, % no vendido **2,3 %**, productividad **8,03**, gráficos y hallazgos.
3. **Cargar datos** → subir `Base_Panaderia_Indicadores_2026-10.xlsx`: **42 turnos**, OEE **71,1 %**.
4. Probar también desde el celular.

6. Dejar evidencia

- Pegar la URL `/exec` en la primera página del informe, junto al enlace del repositorio. En la HU-16, pegarla también en el README (línea «Aplicación (Google Apps Script)»).
- Hacer commit de ese cambio en GitHub con el mensaje `docs: enlace de la app en Google Apps Script`.

Actualizar (HU-16 y cualquier cambio posterior)

El código se cambia siempre en GitHub (`Frontend/`) y después se regenera y se vuelve a pegar el `Index.html`:

```
python Backend/construir_index.py          # regenera Backend/Index.html desde Frontend/
git add Backend/Index.html
git commit -m "build: regenerar Index.html para Apps Script"
```

1. En el editor de Apps Script, reemplazar el contenido de **Index** por el nuevo y guardar.
2. **Implementar → Gestionar implementaciones** → lápiz (editar) → **Versión: Nueva versión** → **Implementar**. **La URL no cambia.**

> No edite `Index` directamente en Apps Script: se perdería en la próxima regeneración y el cambio no quedaría en el repositorio, que fue justamente la observación del docente.

Opción B · Con clasp (desde la terminal)

`clasp` es la herramienta oficial de Google para subir un proyecto de Apps Script desde una carpeta. Necesita Node.js.

1. Activar la API de Apps Script en **script.google.com/home/usersettings** (interruptor «API de Google Apps Script» en *Activado*).
2. En la terminal, dentro del repositorio:

```
npm install -g @google/clasp
clasp login                                          # abre el navegador para autorizar
python Backend/construir_index.py
cd Backend
clasp create --type webapp --title "Tablero Pan Trenza" --rootDir .
clasp push                                           # sube Code.gs, Index.html y
appsscript.json
clasp deploy --description "Entrega final 29 oct."
clasp open                                           # abre el proyecto en el navegador
```

> Si al crear el proyecto `clasp` reemplaza `appsscript.json`, recupere el del repositorio con `git checkout Backend/appsscript.json` antes de `clasp push`.

Luego: **Implementar → Gestionar implementaciones** para copiar la URL `/exec`. `clasp create` crea el archivo `Backend/.clasp.json` con el id del proyecto; súbalo también al repositorio. Para actualizar: `python Backend/construir_index.py` → `clasp push` → `clasp deploy -i <id de la implementación>` (mantiene la URL).

Problemas comunes

Síntoma	Solución
«No se encontró el archivo HTML llamado Index»	El archivo HTML debe llamarse exactamente **Index**.
Pantalla en blanco	Revisar que se pegó **todo** `Index.html` (debe terminar en `</html>`) y guardar antes de implementar.
No aparecen los gráficos	Los gráficos y la lectura de Excel se cargan de **cdnjs.cloudflare.com**: verificar la conexión a internet o que la red no bloquee ese sitio.
Con la cuenta de la universidad no deja elegir «Cualquier usuario»	Las cuentas institucionales a veces lo restringen. Usar una cuenta personal de Gmail.
Los botones de descargar plantilla o datos no descargan	Esos botones abren el archivo en GitHub: funcionan cuando el repositorio es público y la plantilla y los datos ya están en `main` (HU-02 y HU-03).
Exportar CSV no descarga	Algunos navegadores bloquean descargas dentro de Apps Script. En ese caso se usa la versión de GitHub Pages para exportar.
Los cambios no se ven	Falta crear una **nueva versión** en «Gestionar implementaciones», o el navegador guardó la versión anterior: recargar con `Ctrl`+`F5`.